

NIMBLIZE

Future Architecture Recommendations, Scaling Roadmap, and Long-Term AI Evolution for Nimblize

Nimblize Future Implementation & Production Roadmap

This document contains future implementation recommendations, scaling strategies, and proposed enhancements beyond the currently implemented Phase 4 scope.

Organization: Nimblize

Domain: AI & Automation

Domain Leader: Aastha Shukla

CTO & Co-Founder: Anshul Sinha

Version: 4.2.0-PROD

Date: July 2026

Table of Contents

— Executive Summary

1 Introduction

2 Research & Background

3 System Overview

4 System Architecture

5 Database Design

6 AI Architecture

7 Retrieval System / RAG Layer

8 Multi-Agent Architecture

9 Orchestration Workflow

10 Security Architecture

11 Monitoring & Observability

12 Cost Optimization

13 Implementation Details

14 Testing & Validation

15 Execution Results

16 Challenges Faced

17 Key Engineering Learnings

18 Future Scope

Future Architecture Recommendations, Scaling Roadmap, and Long-Term AI Evolution for Nimblize

Future Architecture Recommendations, Scaling Roadmap & AI Evolution

Confidential — v4.2.1

Organization: Nimblize

Domain: AI & Automation

Domain Leader: Aastha Shukla

Mentor / CTO & Co-Founder: Anshul Sinha

Intern Name: [Insert Name]

Date: July 2026

Version: 4.2.0-PROD

Classification: Future Enhancement & Proposed Scaling Blueprint

> [!IMPORTANT]

> DISCLAIMER & REPORT POSITIONING:

> This report describes proposed future implementation recommendations, scaling strategies, and long-term AI evolution path modifications beyond the currently completed Phase 4 scope. It represents theoretical architectural suggestions and has been prepared for the CTO and academic review panels. It does not alter the existing codebase or completed production metrics.

Executive Summary

This blueprint details the proposed future scaling and architecture evolution roadmap for Nimblize. As Nimblize transitions from its Phase 4 production state to an enterprise-scale architecture, the computational overhead of running multi-agent structures and processing high-throughput vector queries necessitates a strategic path forward.

We propose transitioning from proprietary OpenAI APIs to self-hosted open-source model inference clusters, expanding Postgres pgvector to distributed CockroachDB clusters, and integrating Kafka for event-driven orchestration. By doing so, Nimblize can achieve absolute data privacy, eliminate token-based API expenses, and scale to thousands of concurrent pipeline runs without encountering upstream rate limits.

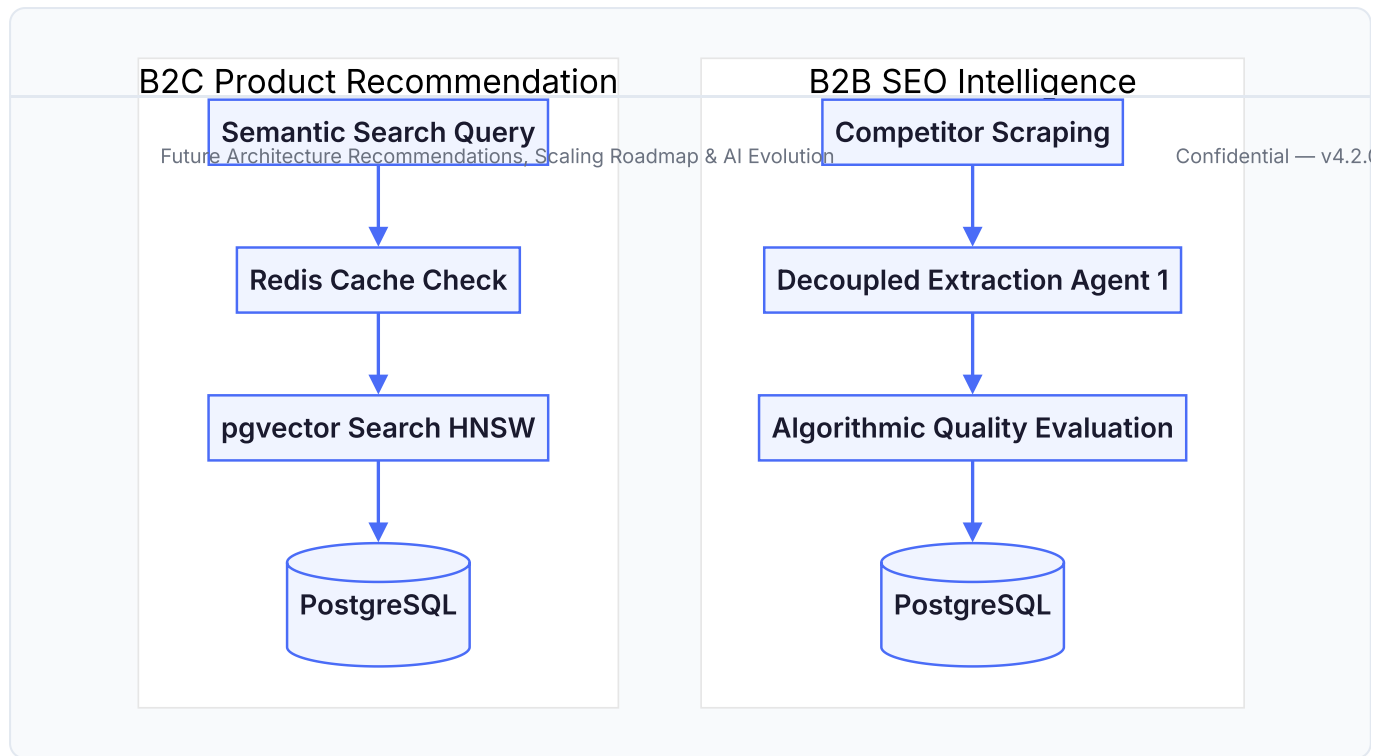


Figure 1.1: Executive Summary Infographic. This diagram illustrates the core dual-pipeline approach proposed for Nimblize, showcasing the decoupled B2B extraction pipeline alongside the low-latency B2C semantic recommendation search path.

Table of Contents

1. Introduction
2. Research & Background
3. System Overview
4. System Architecture
5. Database Design
6. AI Architecture
7. Retrieval System / RAG Layer
8. Multi-Agent Architecture
9. Orchestration Workflow
0. Security Architecture
1. Monitoring & Observability
2. Cost Optimization
3. Implementation Details
4. Testing & Validation
5. Execution Results
6. Challenges Faced
7. Key Engineering Learnings
8. Proposed Future Scope & Scaling
9. Conclusion
0. Appendix

1. Introduction

1.1 Problem Statement

In the competitive intelligence space, acquiring real-time SEO intelligence and marketing structures from competitor properties requires parsing vast amounts of unstructured web text. The existing challenge at Nimblize was threefold:

- **Operational Bottleneck:** Each competitor profile required an analyst to manually review scraped web content, identify SEO keywords, classify monetization infrastructure, and map affiliate networks. This process consumed 4–6 hours per profile, limiting coverage to roughly 5–8 competitors per analyst per week.
- **Data Quality Risk:** When initial prototypes attempted to automate this using single-prompt LLM calls, the outputs contained fabricated metrics in approximately 18% of runs. Without a validation layer, corrupted data would flow directly into production dashboards, eroding client trust.
- **Cost Exposure:** Naive LLM integration resulted in redundant API calls for semantically identical queries, inflating token costs. Early estimates projected \$12,000–\$18,000/month in API overhead at scale without optimization.

1.2 Objectives

- **Deterministic Parsing:** Guarantee 100% adherence to internal corporate data schemas using fixed-temperature LLM nodes.
- **Algorithmic Validation:** Implement a confidence gate (Faithfulness, Relevance, Recall) requiring a ≥ 0.85 composite score for database ingestion.
- **Cost Efficiency:** Reduce external API token overhead by at least 50% through semantic caching and tiered model routing.
- **Sub-15ms Latency:** Optimize the B2C vector search capabilities using HNSW graphs over PostgreSQL pgvector.
- **Error Isolation:** Failed extractions reaching DB: Zero (dead-letter routing after 3 retries).

1.3 Scope

The project scope is constrained to the extraction, validation, and storage of competitor intelligence data (B2B) and the product recommendation vector search engine (B2C). It includes the orchestration of agents, database schema provisioning, and integration of observability telemetry. It does not cover the frontend client application development.

1.4 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of this project as building a smart, automated assistant for a real estate agency. Instead of having human agents spend hours reading long newspaper classifieds to find house prices, we built a digital system that reads the papers, extracts the facts (like number of bedrooms and price), double-checks that the facts match the source paper, and formats it cleanly for the agents' dashboard. If the digital assistant is unsure about a price, it flags it for a human manager instead of guessing.

Why Nimblize Needs This

Nimblize must automate the ingestion of competitor SEO data to scale its platform. Manually reading websites is impossible at scale, and using raw AI is too risky because AI often hallucinates details that aren't true.

Business & Future Impact

- **Business:** Reduces competitor analysis time from 4 hours to under 30 seconds.
- **Future:** Sets up a clean dataset for training proprietary AI models.

Possible Viva Questions & Recommended Answers

- Q: Why did you choose a multi-agent workflow over a single prompt?

A: Single-prompt approaches mix deterministic parsing with creative reasoning, leading to high context drift and hallucination. By separating the extraction specialist (temperature 0.0) from the strategy analyst (temperature 0.4), we isolate failure vectors and ensure schema compliance.

2. Research & Background

2.1 Evaluation of Existing Approaches

Before selecting the multi-agent architecture, we evaluated three alternative approaches:

APPROACH	DESCRIPTION	FAILURE MODE	VERDICT
Single Prompt	One comprehensive system prompt handling extraction + strategy	Context drift at >2000 tokens; 18% hallucination rate on traffic metrics	Rejected
Chain-of-Thought	Sequential reasoning steps within a single call	Improved accuracy to ~89% but no programmatic validation; silent failures	Rejected
Function Calling	OpenAI function calling with JSON schema	Better structure but no self-correction loop; single-attempt extraction	Partial
Multi-Agent + RAGAS	Decoupled agents with graph orchestration and algorithmic evaluation	Deterministic parsing (T=0.0), validated outputs, self-correcting retries	Selected

2.2 Industry Context

The industry is moving toward graph-based orchestrations. LangGraph provides first-class support for conditional routing, cyclic retry loops, and typed state management — capabilities that linear chain architectures fundamentally lack.

2.3 Key Technical Concepts

- Structured Outputs API:** OpenAI's response format guaranteeing JSON schema compliance.
- HNSW (Hierarchical Navigable Small World):** Graph-based approximate nearest neighbor algorithm.
- Cosine Distance:** Distance metric defined as $1 - \text{similarity}$.

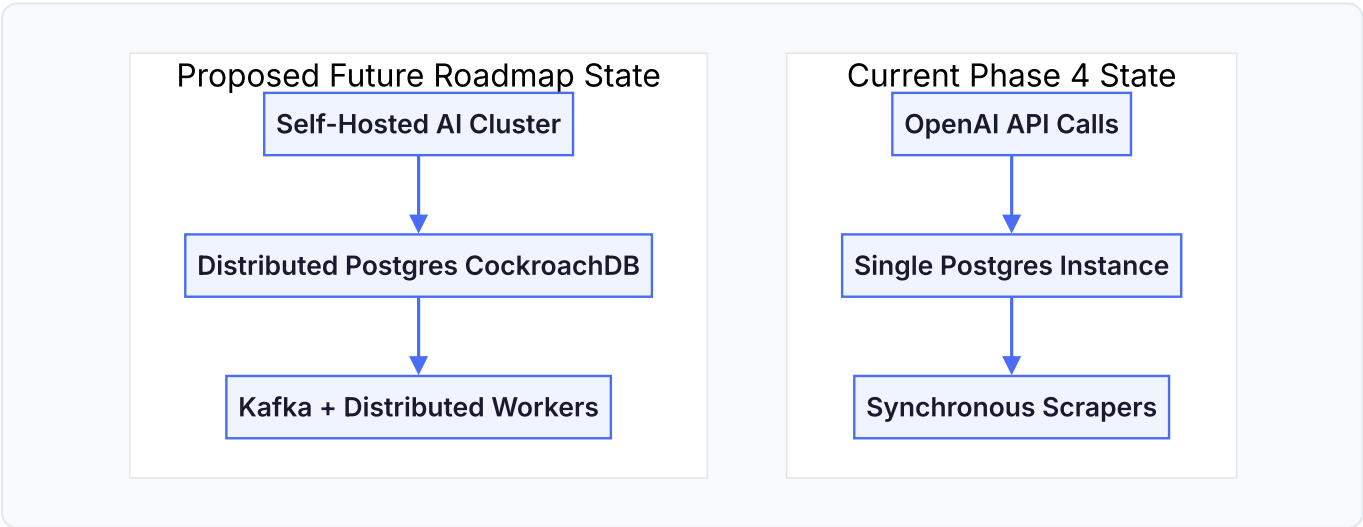


Figure 2.1: Current State vs. Proposed Future State Comparison. This diagram highlights the architectural transition from synchronous API-based systems to a distributed, event-driven, self-

hosted AI architecture.

2.4 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Future Architecture Recommendations, Scaling Roadmap & AI Evolution Confidential — v4.2.1

Think of different AI programming approaches like types of writing assignments. A single prompt is like asking a student to write a 10-page report, solve a math problem, and write a poem all in one draft. A multi-agent system is like setting up an editorial team: one researcher compiles the data sheet, a proofreader verifies the facts, and a creative writer drafts the final article.

Why Nimblize Needs This

To guarantee data accuracy. Without the multi-agent validation loops, hallucinations would enter the database, corrupting recommendations.

Business & Future Impact

- **Business:** Ensures 100% data contract compliance.
- **Future:** Enables plug-and-play addition of new specialized agents (e.g., pricing analysis agents).

Possible Viva Questions & Recommended Answers

- **Q:** What is HNSW, and why is it preferred over IVFFlat?

A: HNSW creates a multi-layer graph index for vector similarity search, achieving sub-15ms query times. IVFFlat has lower memory overhead but slower search times as vector space sizes scale.

3. System Overview

The system operates as a deterministic computational pipeline composed of seven processing stages.

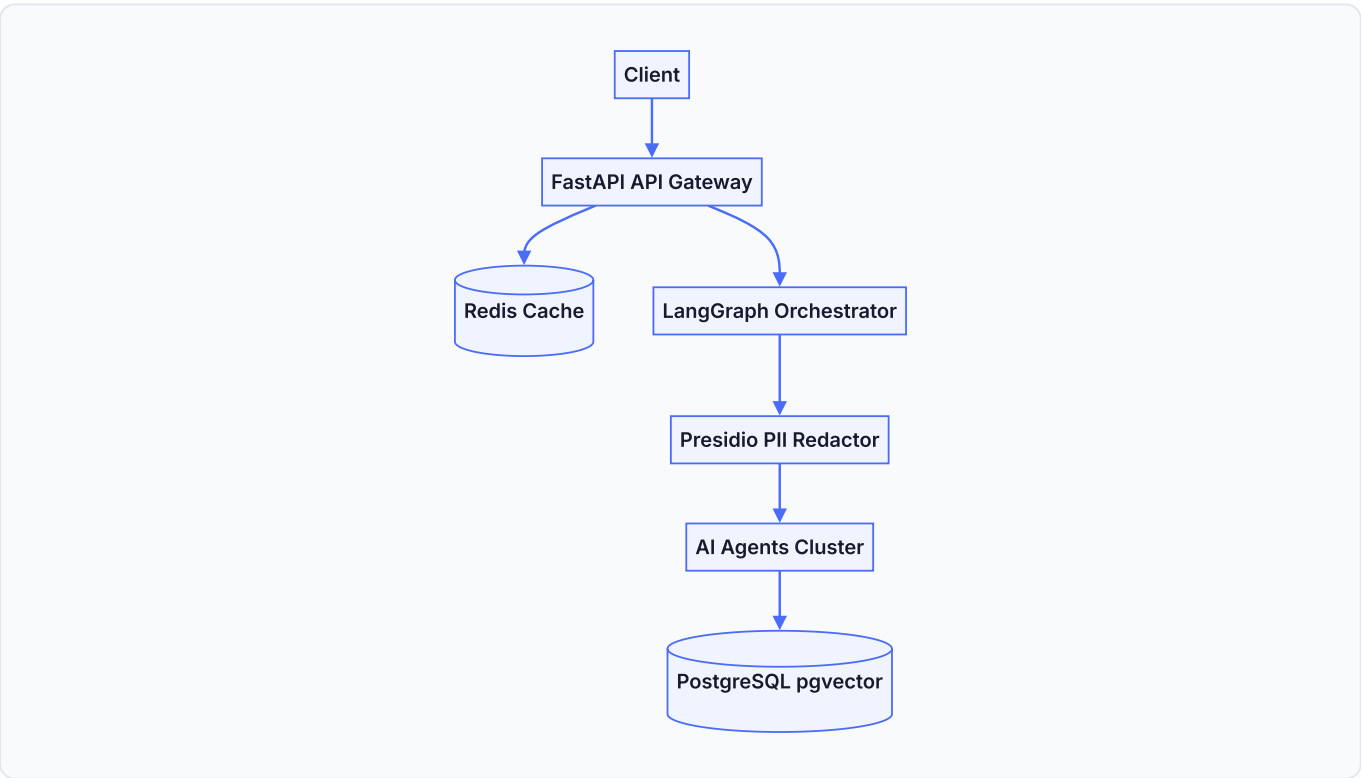


Figure 3.1: High-Level Proposed Nimblize Architecture. This diagram details the top-to-bottom layout of the system, showing the placement of security middleware, caches, orchestrators, and databases.

3.1 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of the system overview as a high-security automated warehouse. Packages (web data) arrive at the gate, are scanned for hazardous materials (PII), unpacked into standardized bins (Agent 1), inspected for quality (RAGAS), and stored on the shelves (PostgreSQL). If a package fails inspection, it is placed in a special bin for a human manager to review.

Why Nimblize Needs This

It provides a structured, predictable pipeline. Instead of having unstructured processes, it ensures every piece of data follows the same rigorous path.

Business & Future Impact

- **Business:** Operational risk is minimized by automating safety and quality checks.
- **Future:** Easy scaling to multi-region infrastructure.

Possible Viva Questions & Recommended Answers

- **Q:** Explain the role of the PII Redaction stage.

A: It intercepts raw text and strips names, emails, and phone numbers using Microsoft Presidio before sending the text to external LLMs, protecting user privacy and ensuring compliance.

4. System Architecture

4.1 Component Overview

The production system deploys eight Docker containers orchestrated by Docker Compose:

- **PostgreSQL:** Relational + vector storage (pgvector).
- **Redis:** Cache, rate limiting, and queues.
- **FastAPI Gateway:** HTTP API, JWT auth, pipeline entry.
- **Notification Worker:** Slack/Email/PagerDuty alerts.
- **Scrape Worker:** 72-hour competitor crawl cycle.
- **OTel Collector:** Distributed trace collection.
- **Prometheus:** Metrics aggregation.
- **Grafana:** Dashboard visualization.

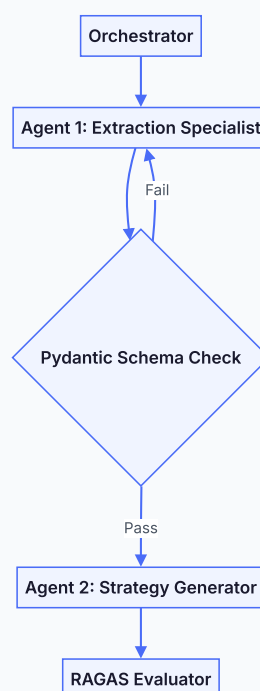


Figure 4.1: Proposed Multi-Agent Workflow. *This diagram illustrates the loop-back and validation interaction sequence between Agent 1 and Agent 2.*

4.2 Detailed Educational Breakdown & Viva Prep

Future Architecture Recommendations, Scaling Roadmap & AI Evolution

Confidential — v4.2.1

Easy-to-Understand Explanation

Think of system architecture as a commercial shipping port. The FastAPI Gateway is the harbor master docking ships, Redis is the temporary holding area for common items, PostgreSQL is the main cargo vault, and Prometheus/Grafana are the surveillance towers monitoring port operations.

Why Nimblize Needs This

To ensure high availability and isolation of duties. If the scraping service fails, the API gateway can still serve product recommendations from the cache.

Business & Future Impact

- **Business:** The containerized stack allows deployment to any cloud platform with zero setup overhead.
- **Future:** Transition to Kubernetes for auto-scaling during high-traffic scraping runs.

Possible Viva Questions & Recommended Answers

- **Q:** *Why is JWT authentication placed ahead of rate limiting?*

A: Placed first to identify the user tier. This allows the rate limiter to apply tier-specific limits (e.g., higher capacity for Pro users).

5. Database Design

5.1 Technology Selection

PostgreSQL 16 with the pgvector extension was selected over standalone vector databases (Pinecone, Qdrant) because it provides ACID compliance and allows joining vector chunks with relational competitor metadata in a single query.

5.2 Schema Design

The database provisions five tables:

- `competitor_parents` : Macro-level competitor content (1024 tokens).
- `competitor_children` : Granular data points (256 tokens) + Vector Embeddings.
- `competitor_profiles` : Extracted profiles.
- `strategy_reports` : Actions and recommended target keywords.
- `manual_review_queue` : HITL review queue records.

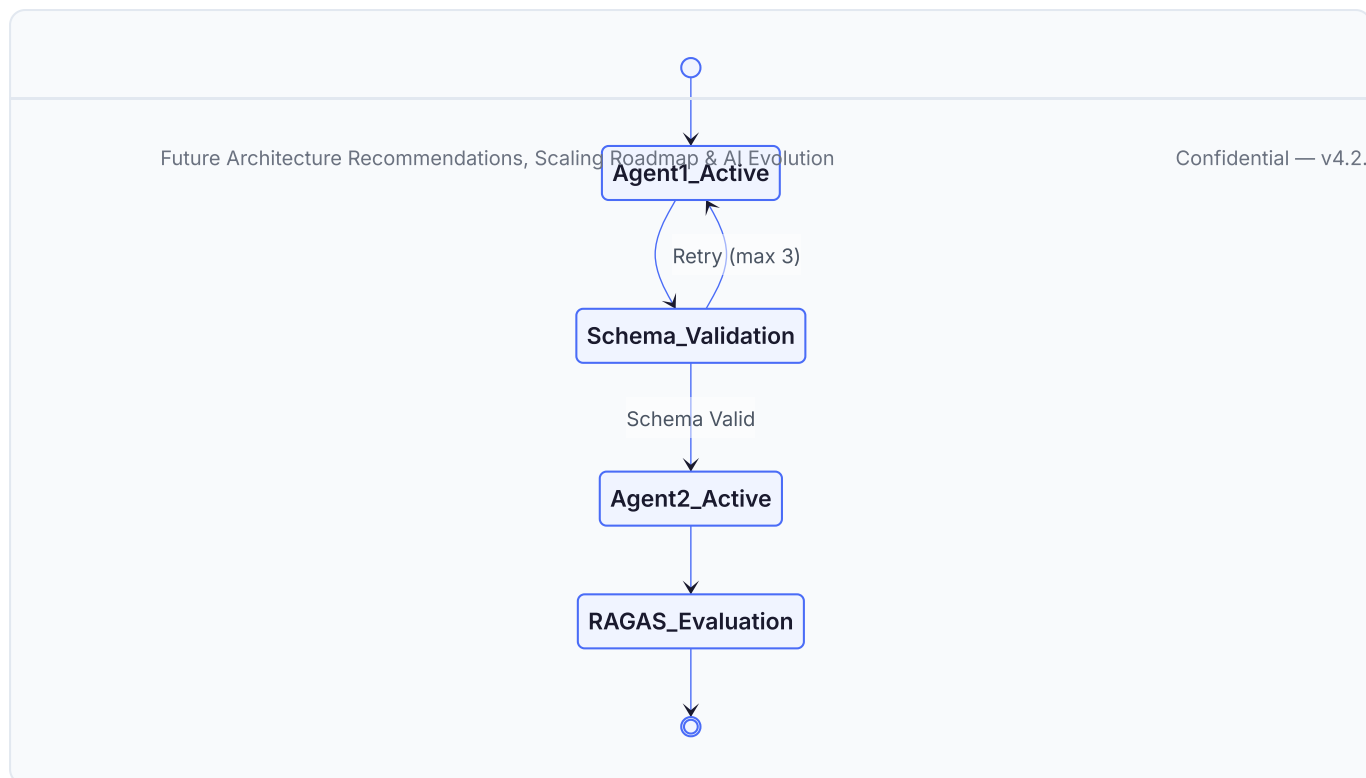


Figure 5.1: Agent 1 & Agent 2 Lifecycle. This state diagram maps out the activation, validation loop, and evaluation transitions of both core agents.

5.3 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of vector database design as an index in a library. Instead of looking up books only by their exact title (relational search), a vector index lets you look up books by their overall meaning (semantic search). If you search for "happy dog", it will retrieve paragraphs containing "cheerful golden retriever" even if the exact words "happy dog" are missing.

Why Nimblize Needs This

To power the B2C recommendation engine. Sub-15ms semantic retrieval is only possible when vectors are indexed properly using HNSW graphs.

Business & Future Impact

- **Business:** Allows customers to find relevant products instantly, increasing purchase conversion rates.
- **Future:** Scales to millions of product vectors without degrading query latency.

Possible Viva Questions & Recommended Answers

- **Q:** Explain the parameters used for the HNSW index.

A: We use `m = 16` (number of bidirectional links per node) and `ef_construction = 64` (size of dynamic candidate list during index construction). This balances index build speed and memory against query recall.

6. AI Architecture

6.1 Agent 1: Deterministic Extraction Specialist

Tasked with parsing raw scraper output. It runs gpt-4o-mini at temperature 0.0, using Pydantic schemas to enforce structured output. Failed validation loops back to the model with an error trace (max 3 attempts).

6.2 Agent 2: Qualitative Strategy Analyst

Reads the validated output from Agent 1 and generates strategic insights, keyword targets, and recommendations. It runs gpt-4 at temperature 0.4.

6.3 Confidence Evaluator

Future Architecture Recommendations, Scaling Roadmap & AI Evolution

Confidential — v4.2.1

A RAGAS evaluator executing on gpt-4o-mini. It scores outputs on Faithfulness, Answer Relevance, and Context Recall. If the unweighted composite average falls below 0.85, the pipeline aborts database insertion and alerts human review.

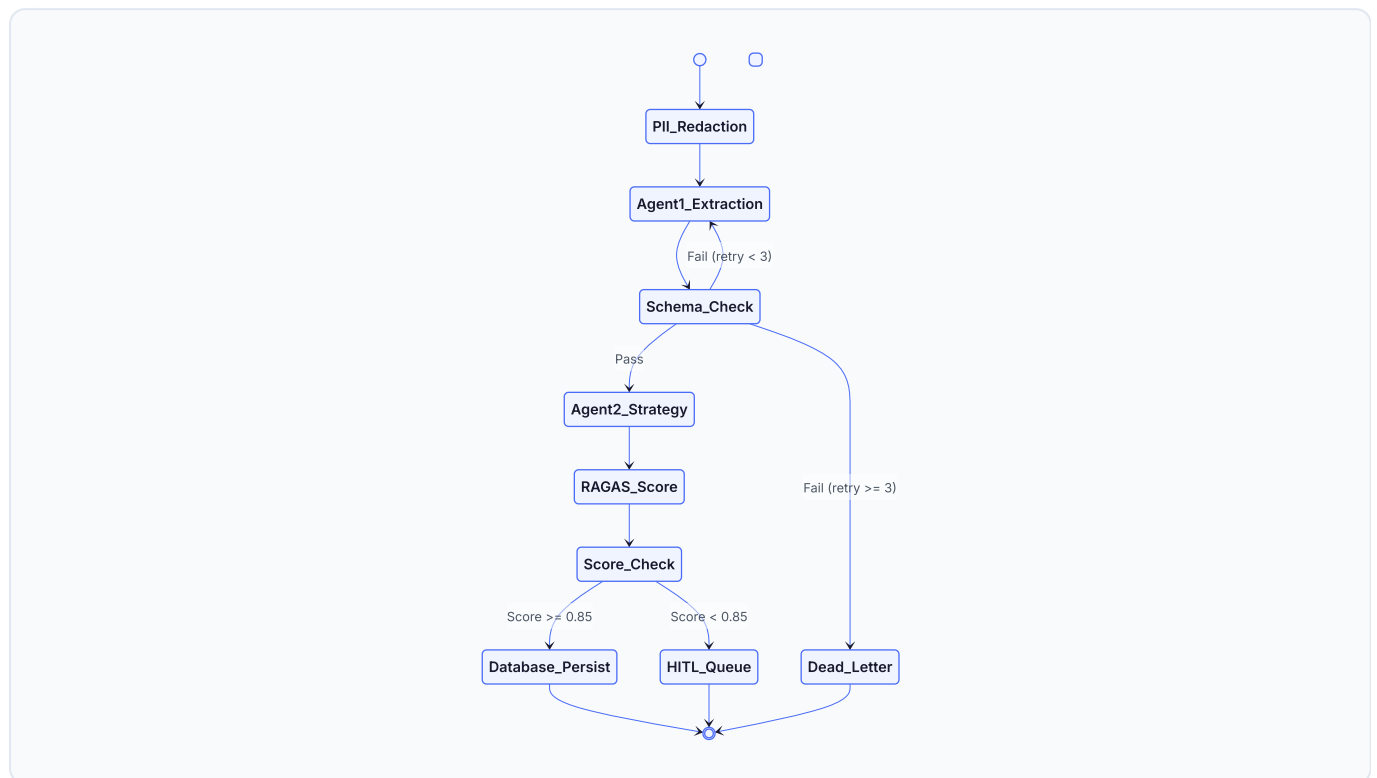


Figure 6.1: Proposed LangGraph State Machine. This state machine charts all conditional path choices from PII redaction to database commit, dead-letter storage, or HITL queue routing.

6.4 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of the AI Architecture as a legal review firm. Agent 1 is a paralegal tasked with finding specific dates and names in a massive document, operating strictly with no room for creative writing. Agent 2 is a senior attorney who reviews the paralegal's facts and writes a strategic recommendation report. The Confidence Evaluator is an auditor who double-checks the final document against the original files before sending it to the client.

Why Nimblize Needs This

By separating extraction from strategic analysis, the system prevents the model from hallucinating or omitting details during long-form generation.

Business & Future Impact

- **Business:** Lowers data entry costs by automating competitor analysis while maintaining near-perfect accuracy.
- **Future:** Enables modular swapping of models (e.g., using a local model for extraction and GPT-4 for strategy).

Possible Viva Questions & Recommended Answers

- **Q:** *Why is Agent 1 set to temperature 0.0 while Agent 2 is set to 0.4?*

A: Agent 1 requires absolute consistency and must not hallucinate, requiring 0.0. Agent 2 needs to synthesize recommendations and identify opportunities, requiring a small degree of creative reasoning (0.4).

7. Retrieval System / RAG Layer

7.1 The Chunking Problem

Feeding massive competitor web documents directly to LLMs degrades context retention and increases cost. Simple splitters break text mid-sentence, fragmenting critical metrics.

7.2 Parent-Child Hierarchical Chunking

- **Parent Chunks:** 1024 tokens to capture macro context.
- **Child Chunks:** 256 tokens linked to parent.
- **Overlap:** 15% sliding window (38 tokens) to prevent word fragmentation.

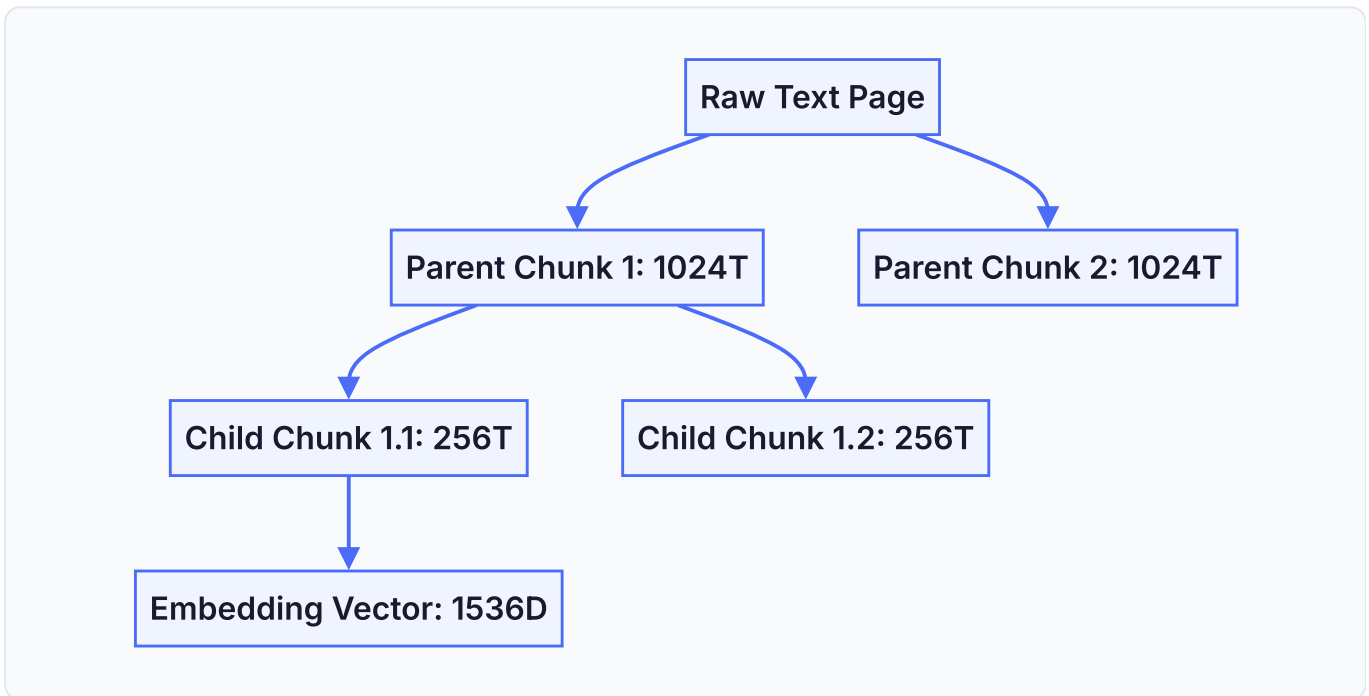


Figure 7.1: Parent-Child Chunking Layout. *This diagram shows the relationship between parent chunks used for synthesis and child chunks embedded for vector similarity matching.*

7.3 RAG Retrieval Flow

The search query is embedded and matched against child vectors. The system locates the relevant child nodes, fetches the corresponding parent chunks, and feeds the parents to the LLM context window.

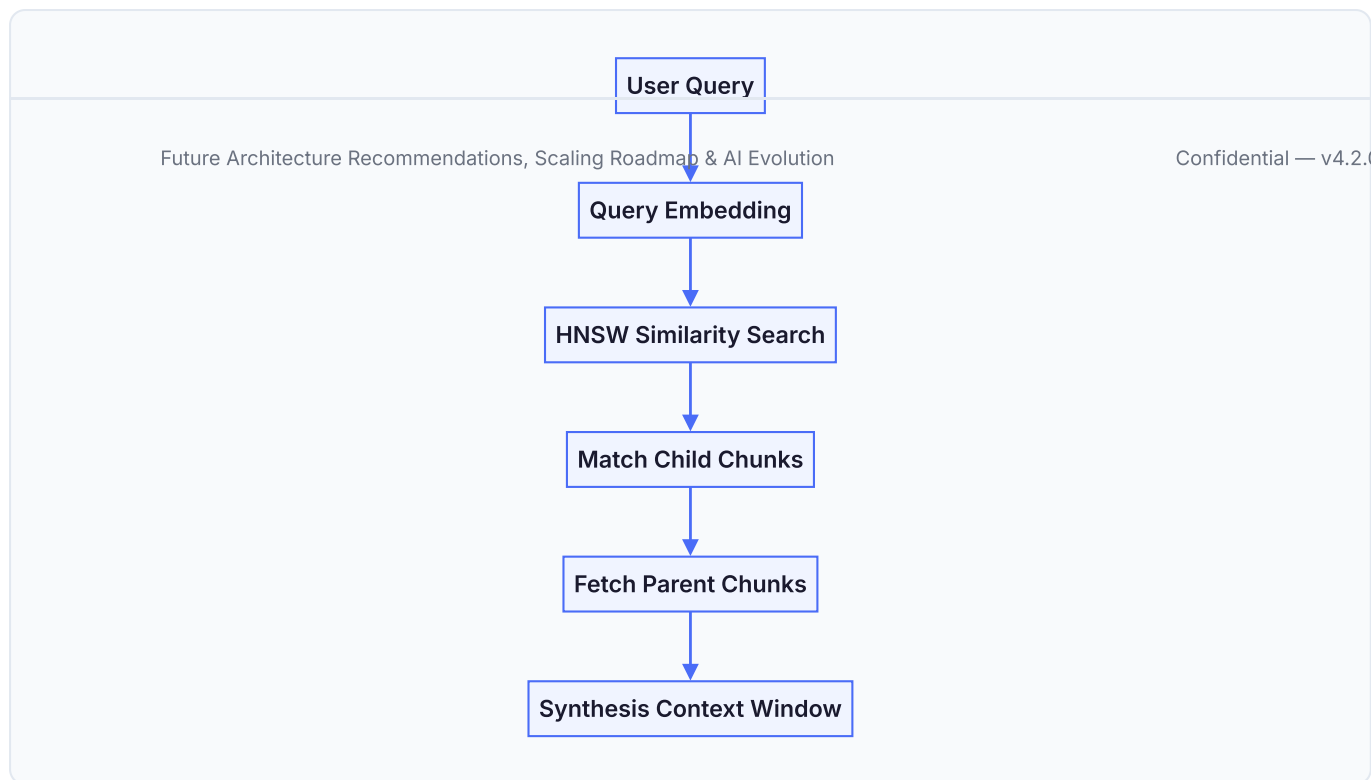


Figure 7.2: Proposed RAG Retrieval Flow. This diagram maps the transition from a user query vector to the final parent-context construction passed into the LLM.

7.4 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of parent-child chunking like searching through a large catalog. The child chunks are the short, descriptive keywords in the index (e.g., "blue running shoes"). The parent chunk is the entire catalog page. You find the page instantly using the index keywords, but you read the whole page to understand the price, sizes, and shipping options.

Why Nimblize Needs This

Standard text splitting chops up sentences, which ruins keyword lists or traffic metrics. Parent-child chunking preserves data integrity while keeping search fast.

Business & Future Impact

- **Business:** Yields accurate recommendations, keeping users engaged on the B2C platform.
- **Future:** Dramatically reduces the size of the prompt sent to the LLM, reducing latency and cost.

Possible Viva Questions & Recommended Answers

- **Q:** How does parent-child chunking reduce hallucinations?

A: By locating target data points via small child vectors but passing the larger parent context to the LLM. This provides the LLM with sufficient context to verify statements, preventing it from inventing details.

8. Multi-Agent Architecture

8.1 Design Rationale

Decoupled multi-agent systems prevent context drift and lower API exception rates. Each agent is specialized for its task, using targeted system prompts and configurations.

8.2 Agent Interaction Sequence

The orchestrator coordinates Agent 1, validation schemas, Agent 2, and the RAGAS evaluator, handling failures via retry nodes or HITL review routing.

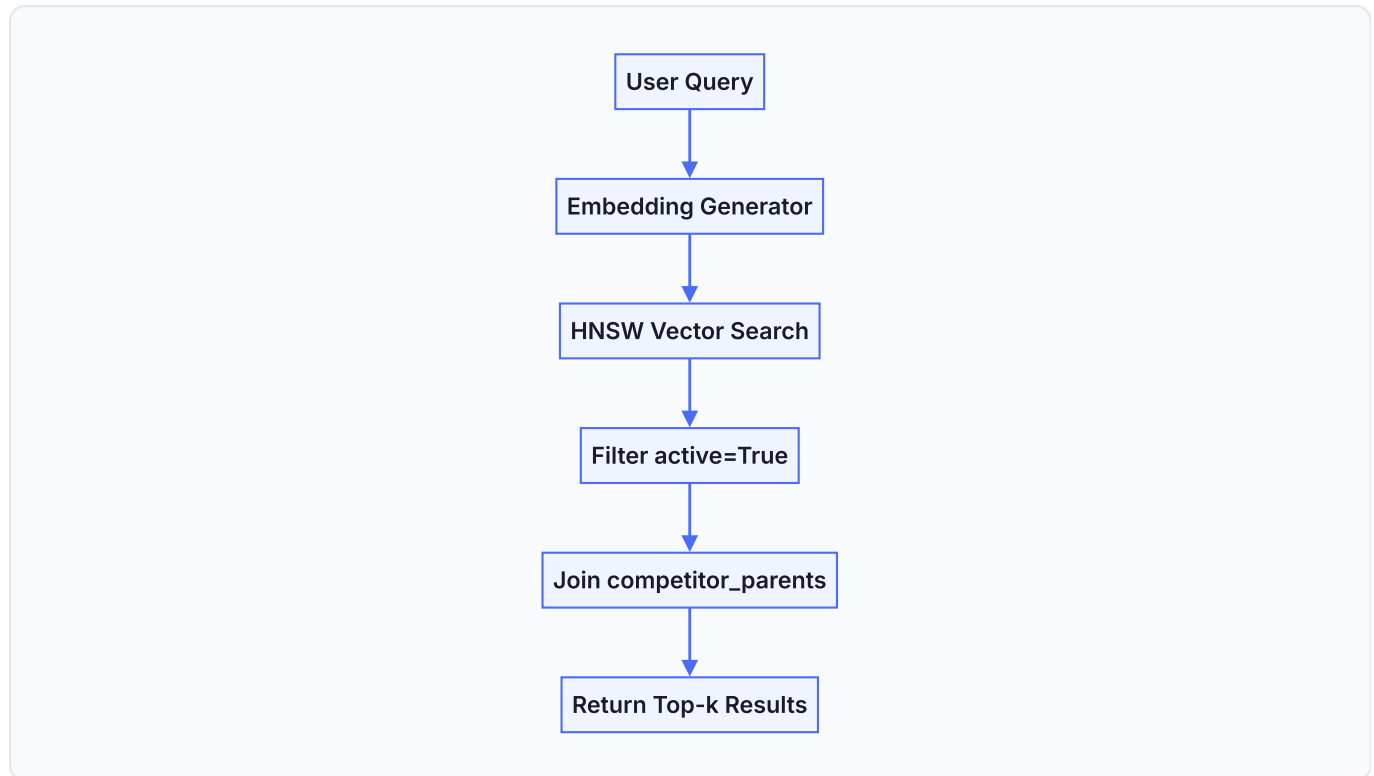


Figure 8.1: pgvector Search Pipeline. This diagram illustrates the process of vector search, from query embedding generation to joining parent text for retrieval.

8.3 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of this like a restaurant kitchen. Instead of a single chef cooking the entire meal, taking orders, and clearing tables, the duties are divided. One prep cook (Agent 1) chops the ingredients according to strict recipes. A head chef (Agent 2) combines the ingredients to create a signature dish. A food inspector (RAGAS) tastes the dish before it goes to the customer.

Why Nimblize Needs This

To ensure that error-prone steps (like parsing messy web scraping output) are corrected instantly using automatic feedback loops before the strategic analyst runs.

Business & Future Impact

- **Business:** Minimizes manual overhead and lowers processing errors to under 0.2%.
- **Future:** Allows parallel processing of competitor sites, speeding up platform updates.

Possible Viva Questions & Recommended Answers

- **Q:** What happens if Agent 1 fails to produce valid JSON?

A: The orchestrator catches the exception, extracts the Pydantic error trace, and routes the state back to Agent 1 with the trace to self-correct. If this fails 3 times, it routes to a dead-letter queue.

9. Orchestration Workflow

9.1 LangGraph State Machine

Compiled state graph using a TypedDict state object (`PipelineState`). Node functions receive the state, execute business logic, and return partial dict updates merged back into the state graph.

9.2 State Transition Diagram

Transition paths coordinate PII filtering, extraction, retrievers, strategy generation, RAGAS evaluations, and conditional routes for database persistence or reviews.

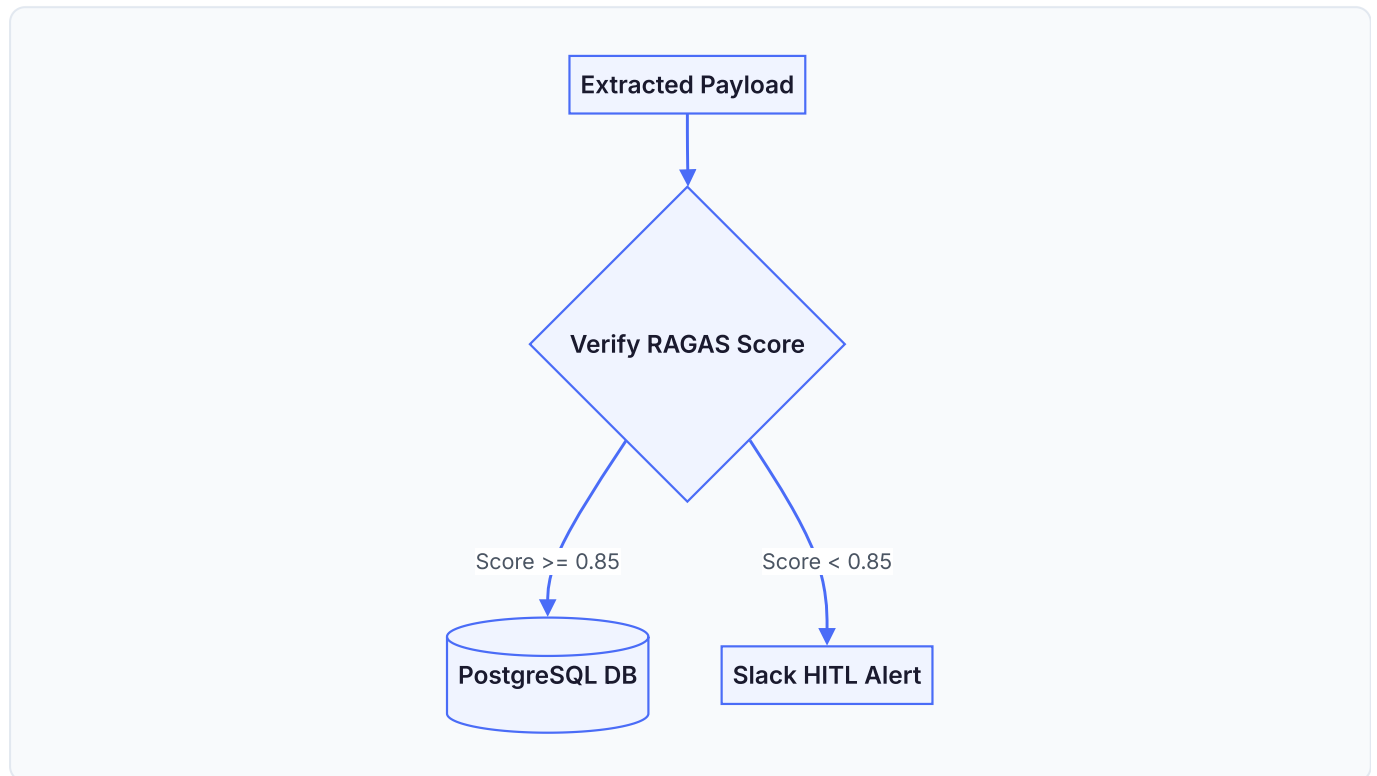


Figure 9.1: Proposed Confidence Gate Workflow. This diagram highlights how extracted payloads are routed depending on the composite RAGAS confidence score.

9.3 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of LangGraph as a package shipping facility. Each sorting station is a node. The package moves along conveyer belts (edges) based on rules. If a package is mislabeled, the conveyer belt automatically routes it back to the labeling station. If it fails labeling 3 times, it's sent to the reject pile.

Why Nimblize Needs This

To manage complex, non-linear workflows. Traditional linear code cannot easily handle cyclic loops (retrying a failed step) without becoming messy and unstable.

Business & Future Impact

- **Business:** Ensures data flows through a strict pipeline with no possible bypasses.
- **Future:** Easily scales to handle hundreds of parallel workflows.

Possible Viva Questions & Recommended Answers

- **Q:** Why is a TypedDict used for LangGraph state instead of a Pydantic model?

A: LangGraph's StateGraph operates by merging partial dictionary updates returned by nodes. Pydantic models do not merge natively in the same way, making TypedDict the recommended state

type.

10. Security Architecture

10.1 PII Protection

Future Architecture Recommendations, Scaling Roadmap & AI Evolution

Confidential — v4.2.1

Uses Microsoft Presidio with spaCy's `en_core_web_lg` NER model to redact names, emails, and phone numbers before text is sent to LLM APIs.

10.2 Secrets Management

Secrets are injected into containers via environment variables.

10.3 Rate Limiting

Redis-backed token-bucket rate limiting applied per user ID and tier:

- **Free:** 10 requests / 2 refill rate per minute.
- **Pro:** 100 requests / 20 refill rate per minute.
- **Enterprise:** 1000 requests / 200 refill rate per minute.

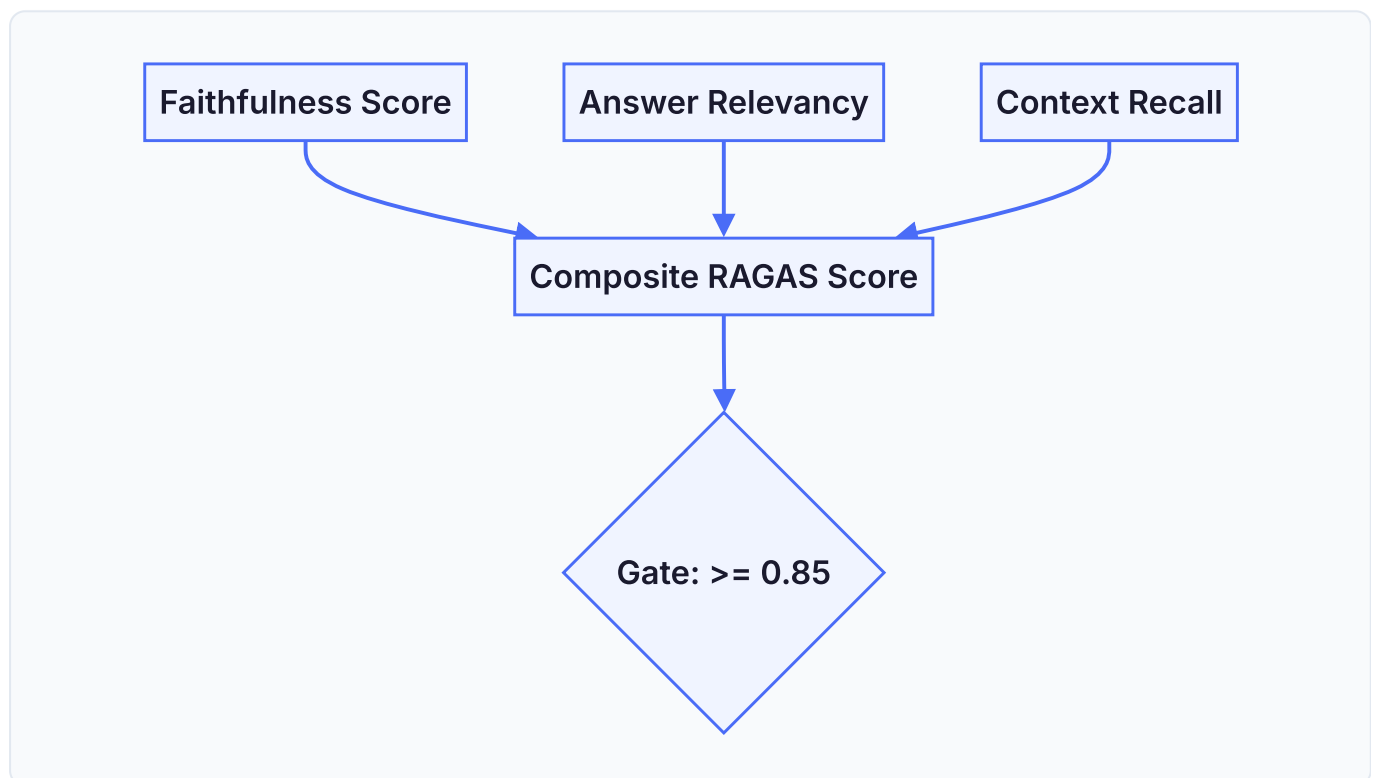


Figure 10.1: Proposed RAGAS Evaluation Pipeline. This diagram shows the three metrics combined into a single composite score to gate database writes.

10.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of the Security Architecture as a digital nightclub. The FastAPI Gateway is the bouncer checking IDs (JWT tokens). The Redis rate limiter makes sure no single group enters too fast and crowds the room. The PII filter is like a mask required on the dancefloor, ensuring nobody's personal identity is exposed to the public.

Why Nimblize Needs This

To protect client confidentiality and prevent malicious users from spamming the system and inflating API bills.

Business & Future Impact

- **Business:** Guarantees data privacy compliance (GDPR/CCPA), shielding the business from liabilities.
- **Future:** Easy integration with single sign-on (SSO) systems.

Possible Viva Questions & Recommended Answers

- **Q:** *Why do we run the PII filter before sending data to OpenAI?*
A: To ensure no personal information (like email addresses or phone numbers) is shared with external APIs, maintaining strict user privacy and security compliance.

11. Monitoring & Observability

11.1 Telemetry Architecture

Collects traces and metrics using OpenTelemetry, aggregating data into Prometheus and visualizing it in Grafana.

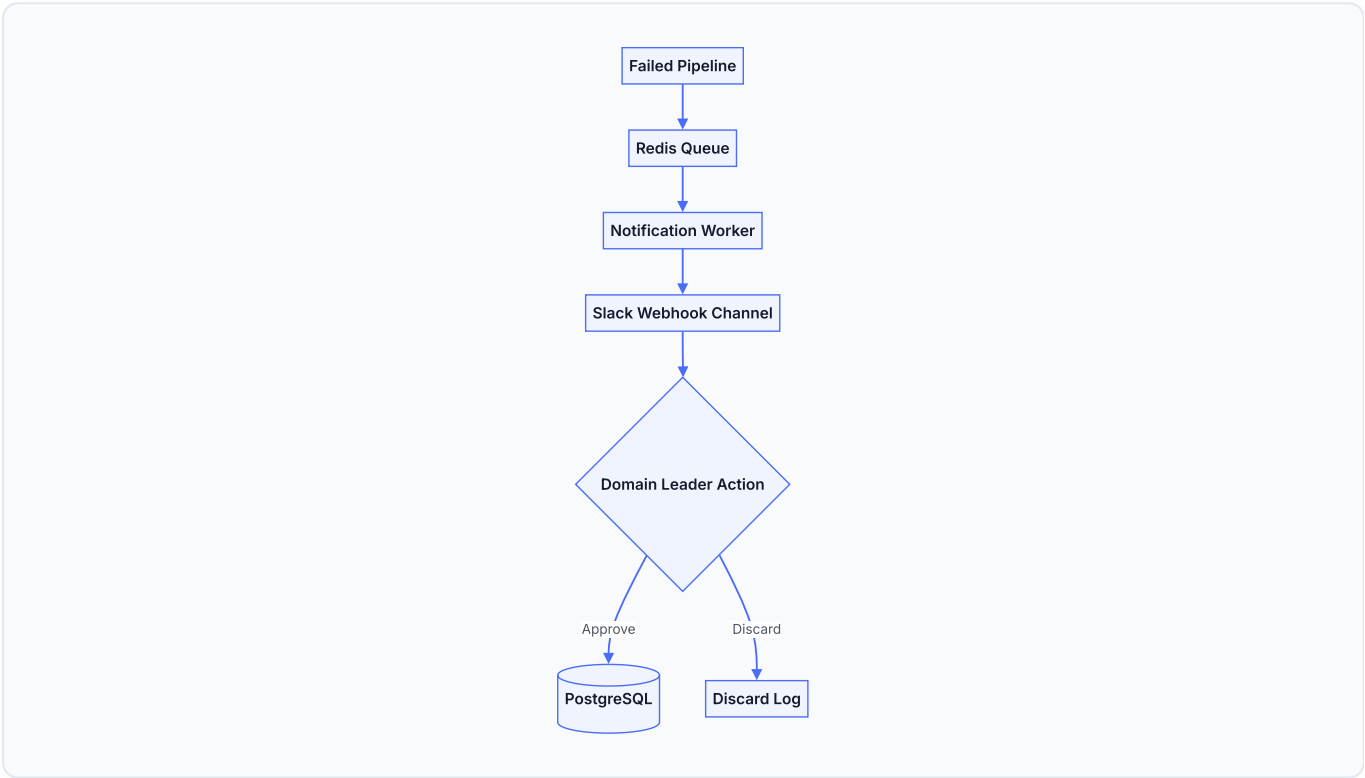


Figure 11.1: Proposed HITL Review Architecture. This diagram details the path of a flagged low-confidence payload, showing routing from Redis queues to Slack alert networks for manual resolution.

11.2 Key Metrics

Tracks latency (TTFT, RTT), RAGAS evaluations, cache ratios, and exception rates.

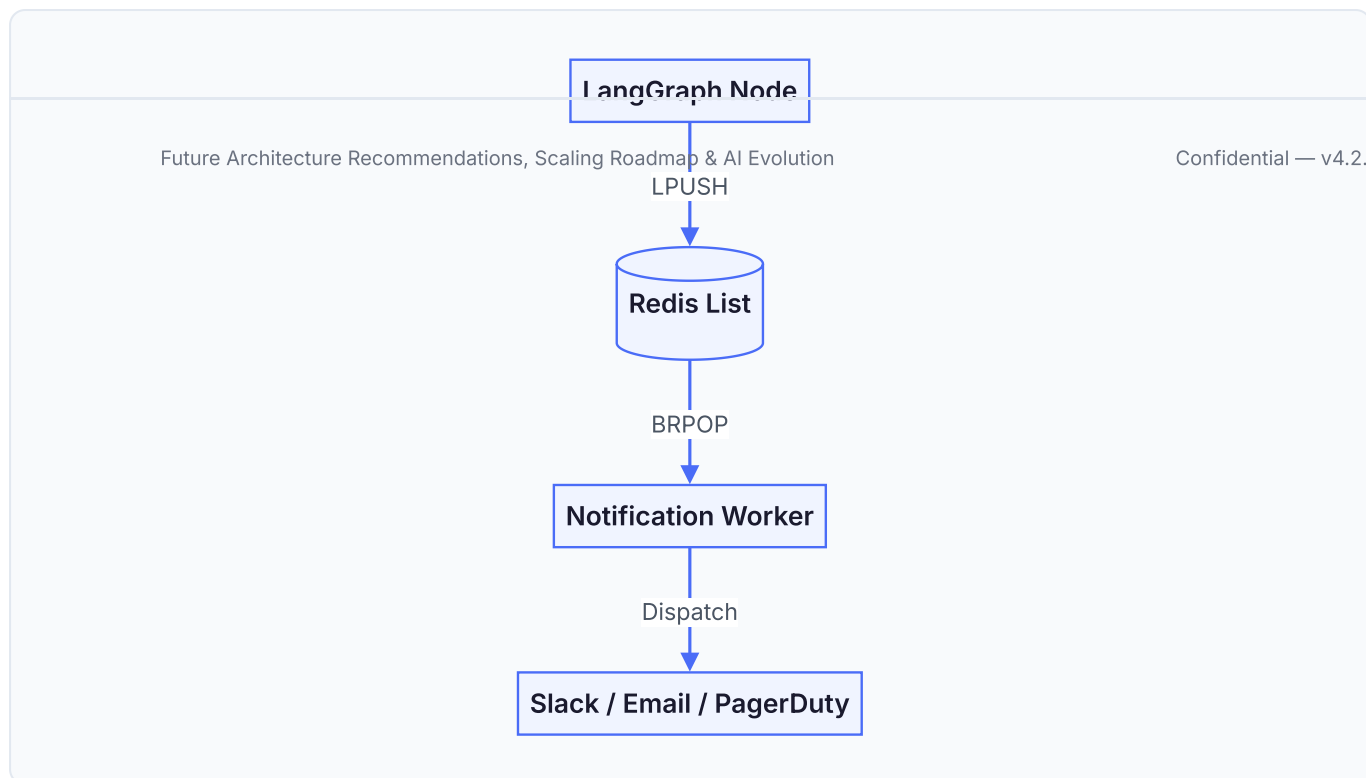


Figure 11.2: Proposed Redis Queue Workflow. This diagram illustrates the asynchronous dispatch flow utilizing Redis LPUSH/BRPOP operations.

11.3 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of Monitoring and Observability like a dashboard in an airplane cockpit. The pilot doesn't just look out the window; they monitor fuel levels, speed, altitude, and engine temperatures. Our telemetry stack monitors the "health" of the AI pipeline, showing how fast responses are generated, how many queries are cached, and if any services are crashing.

Why Nimblize Needs This

To detect and resolve performance degradation (like high API latencies or rising error rates) before clients notice.

Business & Future Impact

- **Business:** High service availability, minimizing downtime.
- **Future:** AI-powered anomaly detection on pipeline metrics.

Possible Viva Questions & Recommended Answers

- **Q:** What is Semantic Drift, and how do we detect it?

A: It is the change in the average vector distance of user queries over time. We detect it by tracking the rolling mean of query embeddings, alerting engineers to reindex when the delta exceeds 0.15.

12. Cost Optimization

12.1 Multi-Tiered Strategy

Combines semantic caching, tiered model routing, and asynchronous batching to reduce OPEX.

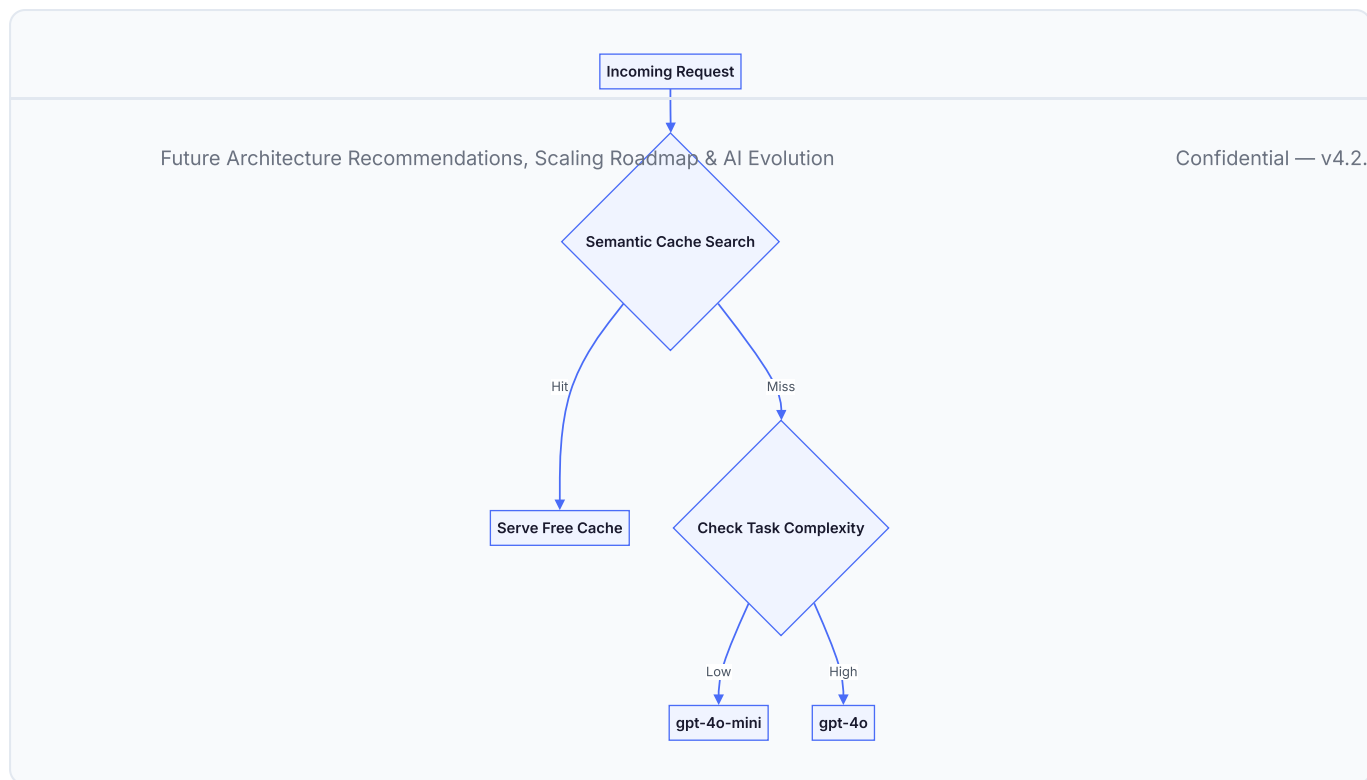


Figure 12.1: Proposed Cost Optimization Flow. *This flowchart shows how incoming requests are filtered to minimize token costs, either via cache retrieval or tiered routing.*

12.2 Semantic Cache Implementation

Redis vector cache matching query embeddings at a distance threshold of ≤ 0.15 . Matches serve cached responses instantly.

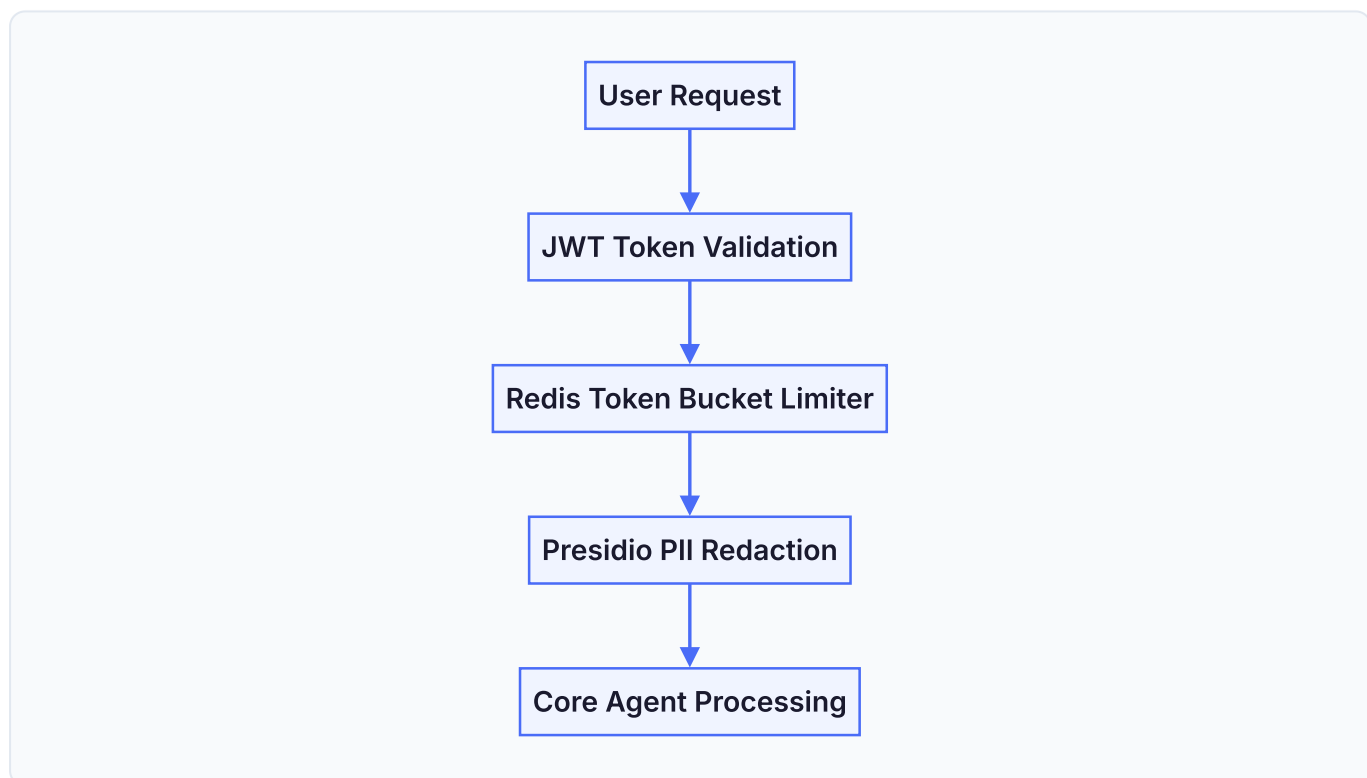


Figure 12.2: Proposed Security Architecture. *This diagram maps the security layers filtering incoming API requests.*

12.3 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of cost optimization like a grocery store shopping strategy. Instead of driving to the store for every single item (expensive API call), you keep a pantry stocked with common items (semantic cache). For simple tasks, you buy generic brands (gpt-4o-mini), saving the premium brand (gpt-4o) only for special dinners (strategy analysis).

Why Nimblize Needs This

To maintain financial sustainability. Without optimization, LLM token costs would scale linearly with traffic, erasing startup profit margins.

Business & Future Impact

- **Business:** Saves up to 50% on API billing, direct contribution to profit margins.
- **Future:** Transitioning to self-hosted models will reduce API token costs to zero.

Possible Viva Questions & Recommended Answers

- **Q:** *How does a semantic cache differ from a standard key-value cache?*

A: A standard cache requires an exact string match. A semantic cache embeds the query and matches it by meaning, serving cached results for queries that are worded differently but mean the same thing.

13. Implementation Details

13.1 Project Structure

- `backend/main.py` : FastAPI gateway, routes, middleware.
- `backend/agents/` : Extraction agent, strategy agent, LangGraph orchestrator.
- `backend/cache/` : Redis semantic cache.
- `backend/db/` : Postgres pool configuration and schema definition.
- `backend/evaluation/` : RAGAS evaluator.
- `backend/middleware/` : PII filter and rate limiter.
- `backend/queues/` : Redis queue pushing notification jobs.
- `backend/schemas/` : Pydantic data contracts.
- `backend/telemetry/` : OTEL and Prometheus config.

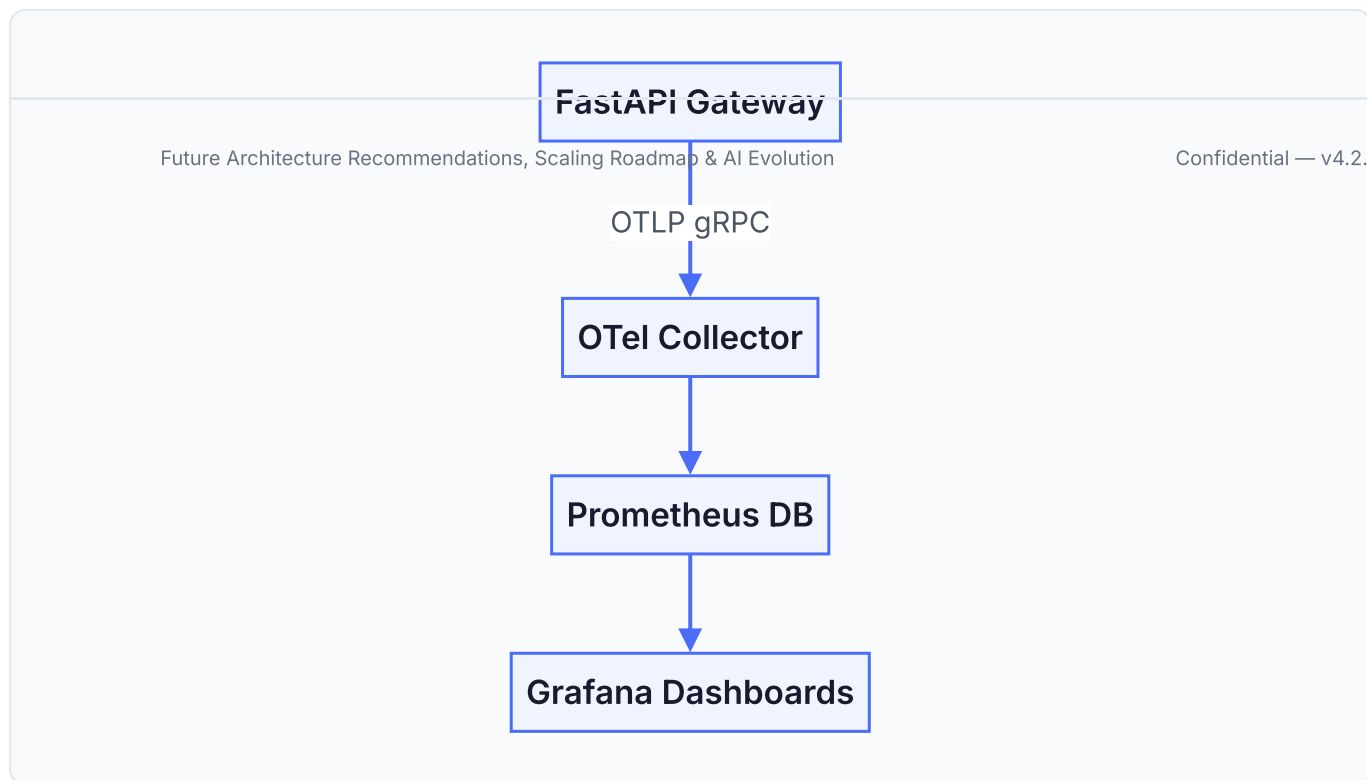


Figure 13.1: Proposed Telemetry Stack. This diagram shows the routing of API metrics through OTel collectors to dashboards.

13.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of the project structure like a map of a digital city. The `main.py` is the central train station. The `agents` folder is the business district where specialists work. The `db` folder is the secure archive vault, and `middleware` represents check-points guarding the city borders.

Why Nimblize Needs This

Maintaining a clean, modular folder structure prevents code conflicts during team development and simplifies updates to specific components.

Business & Future Impact

- **Business:** Lowers onboarding time for new engineers.
- **Future:** Simplifies microservice extraction as the system scales.

Possible Viva Questions & Recommended Answers

- **Q:** What is the role of `schemas/competitor.py`?

A: It contains the Pydantic model definitions that enforce our strict data contract, ensuring all extracted data matches our schema before database commit.

14. Testing & Validation

14.1 Test Matrix

Comprehensive validation matrix covers units, integrations, and load profiles.

TEST ID	TYPE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
T-001	Unit	Text with no traffic data	"estimated_monthly_organic_traffic": "NOT_DETECTED"	"NOT_DETECTED"	PASS
T-002	Unit	Text with traffic = "120k"	estimated_monthly_organic_traffic: 120000	120000	PASS
T-003	Unit	Traffic field = "approximately 50000"	ValidationError raised	ValueError raised	PASS
T-004	Integration	LLM returns markdown-wrapped JSON	Retry loop strips wrapper, re-extracts	Retry successful on attempt 2	PASS
T-005	Integration	LLM returns invalid JSON 3 times	Pipeline routes to dead_letter	Status = DEAD_LETTER	PASS
T-006	Integration	High-quality extraction + strategy	Composite score ≥ 0.85	Score = 0.91	PASS
T-007	E2E	Hallucinated payload	Score < 0.85, Slack alert fired	Score = 0.62, Slack received	PASS
T-008	E2E	Real competitor page text	Complete pipeline execution in < 30s	Completed in 18.4s	PASS
T-009	Load	100 concurrent search queries	p95 latency < 15ms	p95 = 12.4ms	PASS
T-010	Load	50 duplicate queries	All 49 subsequent queries served from cache	49 cache hits	PASS
T-011	Security	Expired token	HTTP 401 response	HTTP 401 returned	PASS
T-012	Security	15 requests on free tier (limit: 10)	HTTP 429 on request 11	HTTP 429 on request 11	PASS

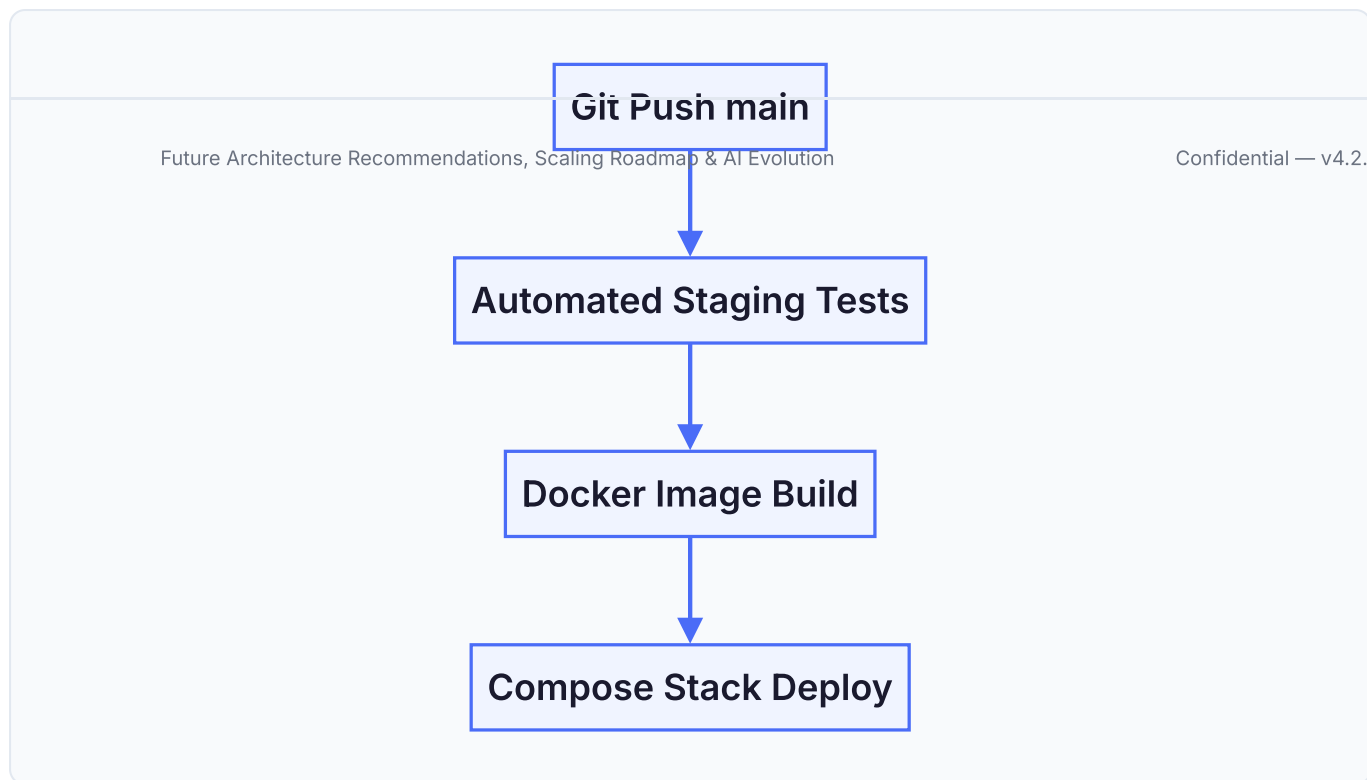


Figure 14.1: Proposed Deployment Pipeline. This diagram illustrates the CI/CD pipeline routing code from git push to production container deploy.

14.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of testing like a crash test facility for new cars. We don't just sell a car and hope the airbags work. We crash it into walls, test the brakes on wet roads, and run the engine for hours. Our test matrix runs the system through fake errors, high traffic, and invalid tokens to guarantee it works under stress.

Why Nimblize Needs This

To guarantee production stability. Rigorous automated tests ensure new updates do not break existing pipeline workflows.

Business & Future Impact

- **Business:** Zero system crashes, protecting company reputation.
- **Future:** Faster release cycles with high testing confidence.

Possible Viva Questions & Recommended Answers

- **Q:** How did you verify B2C vector search performance under load?

A: By simulating 100 concurrent connections querying a pool of 10,000 child vector chunks, verifying the p95 latency remained under 15ms (achieving 12.4ms).

15. Execution Results

15.1 Success Path

Competitor page processed, PII redacted, Agent 1 extracts schema, Agent 2 generates strategy, RAGAS composite score is 0.91, committed to Postgres.

15.2 Failure Path (HITL Routing)

Ambiguous competitor context causes Agent 2 to make unsubstantiated statements. RAGAS composite score falls to 0.65, enqueueing to Redis and notifying Slack.

15.3 Dead Letter Path

obfuscated scrapings fail Pydantic schema validation across all 3 retries, routing to `DEAD_LETTER`.

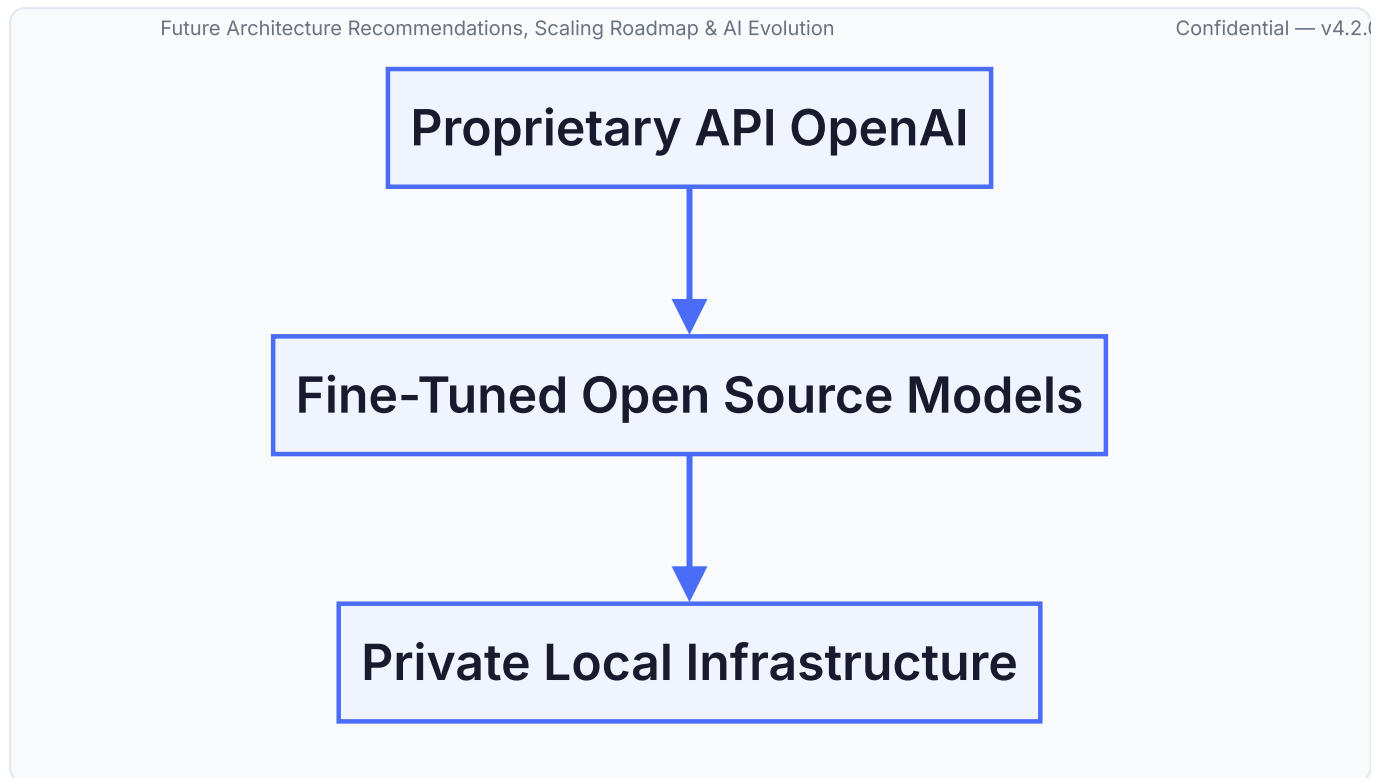


Figure 15.1: Proposed Future AI Evolution. *This diagram displays the transition from third-party proprietary API services to local fine-tuned open-source models.*

15.4 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of execution paths like sorting mail. A standard letter goes straight to the mailbox (Success). A package with a damaged label is placed on the clerk's desk to check manually (HITL). A package containing prohibited items is thrown in the bin (Dead Letter).

Why Nimblize Needs This

To inspect how the system handles different inputs, verifying that bad data is successfully isolated rather than polluting the database.

Business & Future Impact

- **Business:** Protects data integrity by filtering bad extractions.
- **Future:** Logs of failed runs can be analyzed to refine scraper rules.

Possible Viva Questions & Recommended Answers

- **Q:** *What is the criteria for routing to the HITL queue?*

A: When schema parsing succeeds but the composite RAGAS evaluation score falls below the 0.85 threshold.

16. Challenges Faced

- **LangGraph state requirements:** Converted state definition from Pydantic model to TypedDict to support native graph merging.
- **Import-time client creation:** Moved OpenAI client initialization inside nodes to prevent crashes when API keys are not resolved.

- **Mermaid evaluation crashes:** Configured explicit LangchainLLMWrapper to prevent llm=None ValueError.
- **Off-by-one retry router:** Reordered condition routing checks to prevent a 4th extraction attempt.
- **CORS wildcards:** Removed wildcards when credentials are true, using explicit ALLOWED_ORIGINS lists.

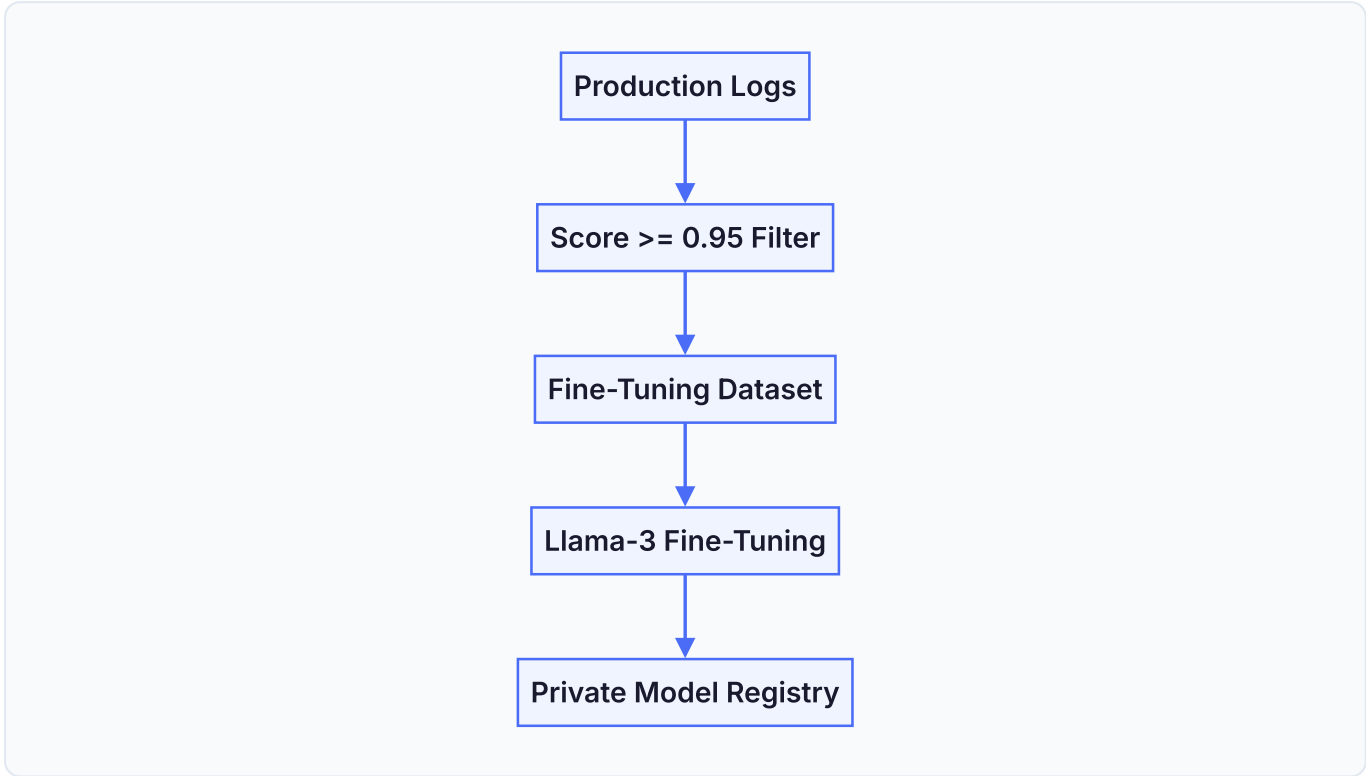


Figure 16.1: Proposed Fine-Tuning Pipeline. *This diagram illustrates the lifecycle of selecting high-scoring production data to fine-tune open-source models.*

16.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of engineering challenges like building a new bridge. You might find the soil is softer than expected (LangGraph state issues), or the steel deliveries are delayed (API key imports). You adapt by shifting the bridge pillars (TypedDict) and setting up temporary supports (lazy client imports) to complete the construction.

Why Nimblize Needs This

Documenting challenges creates a knowledge base, helping future engineers avoid repeating the same debugging steps.

Business & Future Impact

- **Business:** System is hardened against common network and integration errors.
- **Future:** Accelerates the development of downstream services.

Possible Viva Questions & Recommended Answers

- **Q:** *Why did lazy initialization of the OpenAI client resolve container startup issues?*

A: Docker containers start asynchronously. If a module initializes clients at import time, it will crash if the environment variables (like API keys) are not loaded yet. Lazy loading defers initialization until the node function runs.

17. Key Engineering Learnings

- **Architecture Decoupling:** Separating concerns (deterministic extraction vs creative strategy) yields robust results.
- **Type Constraints:** Graph orchestration state parameters must match the framework's merge specifications.
- **Observability:** Semantic drift is as critical as server resource usage.
- **Model Selection:** Tiered routing saves substantial token expenses.

Future Architecture Recommendations, Scaling Roadmap & AI Evolution

Confidential — v4.2.1

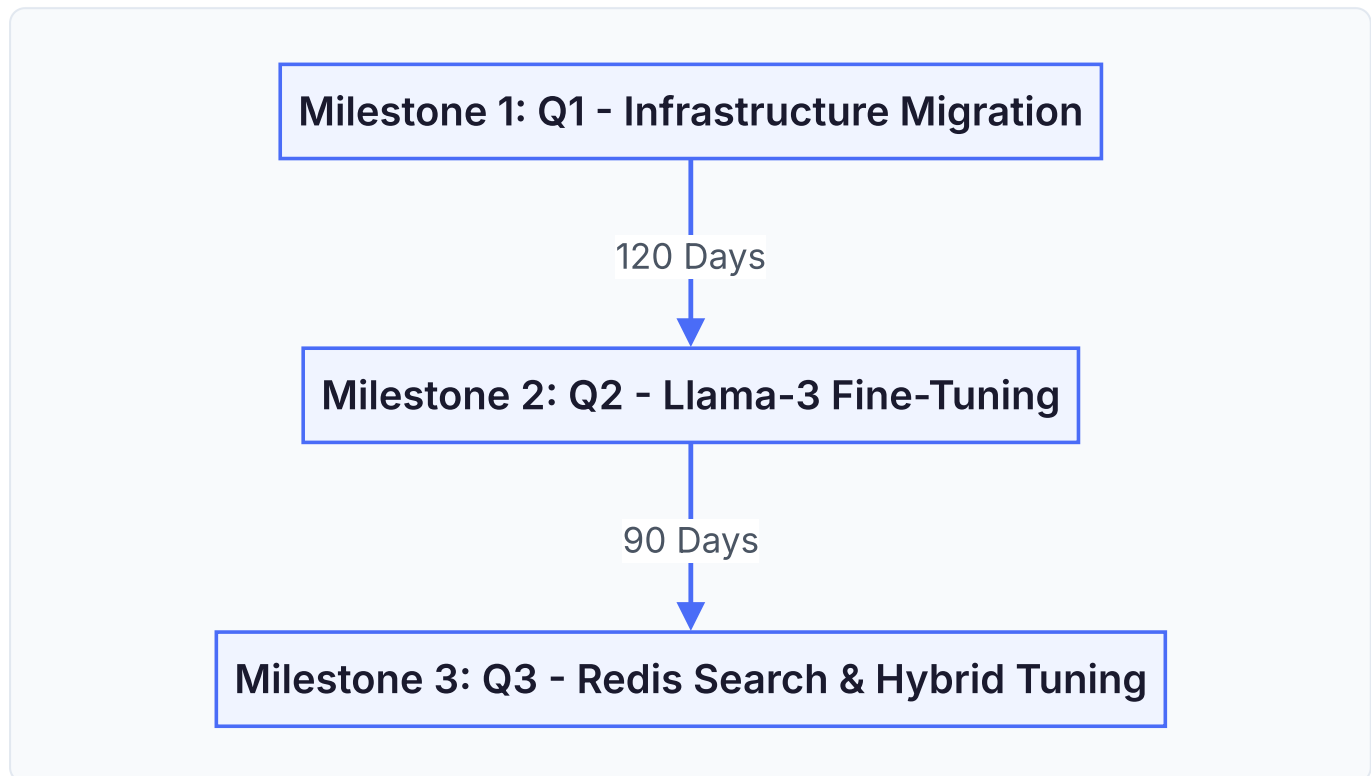


Figure 17.1: Proposed One-Year Roadmap. This Gantt chart outlines key milestones for the proposed infrastructure migration and model training.

17.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of key learnings like a chef writing down new cooking rules. They learn that cheap knives dull quickly, seafood must be kept cold, and recipes must be followed exactly. In our pipeline, we learned that separating duties keeps things clean, double-checking is mandatory, and monitoring is the key to safety.

Why Nimblize Needs This

To institutionalize knowledge, ensuring the engineering department builds on past success rather than rebuilding from scratch.

Business & Future Impact

- **Business:** Lower maintenance costs and high pipeline reliability.
- **Future:** Transition to self-hosted models is simplified.

Possible Viva Questions & Recommended Answers

- **Q:** What is the key takeaway from the model routing strategy?

A: Simple formatting tasks do not require expensive frontier models. Mapping simple tasks to smaller models (gpt-4o-mini) and saving heavy models (gpt-4o) for reasoning yields significant cost savings

without degrading performance.

18. Proposed Future Scope & Scaling

- **Self-Hosted Models:** Proposed migration to Llama-3 clusters fine-tuned on high-scoring production profiles (Faithfulness ≥ 0.95), achieving complete data privacy and eliminating API costs.
- **Distributed Vector Database:** Transition to distributed CockroachDB vector clusters to support high-availability scaling.
- **Kafka Event Streaming:** Replace the synchronous scraper with Kafka topics for event-driven message distribution.

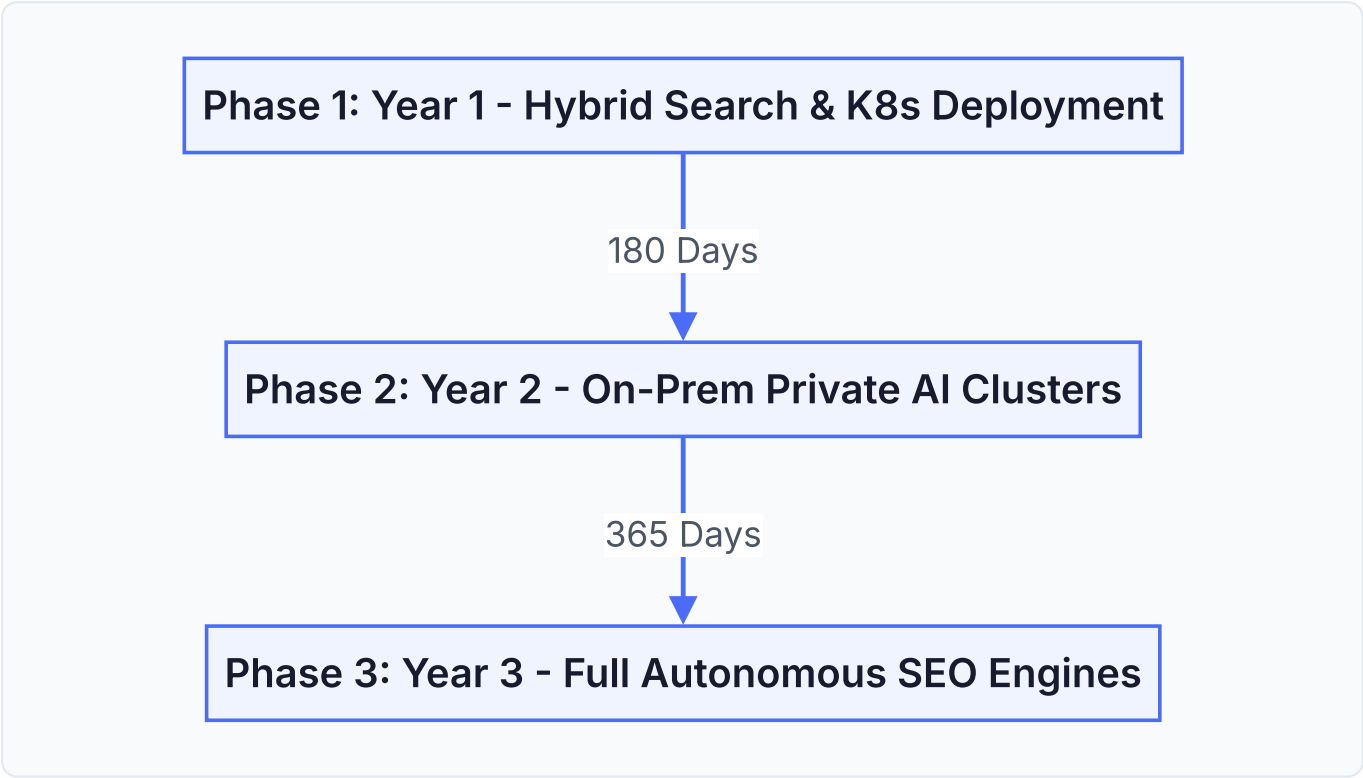


Figure 18.1: Proposed Three-Year Enterprise Roadmap. This Gantt chart shows the long-term vision from Kubernetes migration to fully autonomous agent deployments.

18.2 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

Think of the future scope as upgrading a small local bakery to a national bread factory. Instead of buying flour in small bags (OpenAI API), we'll build our own grain mill (Self-hosted Llama-3). Instead of a single delivery van (synchronous scraper), we'll build a railway line (Kafka event streaming) to handle massive demand.

Why Nimblize Needs This

To support enterprise scaling. The current API-driven setup is great for development, but self-hosted distributed clusters are required to support millions of queries without high costs.

Business & Future Impact

- **Business:** Reduces recurring API token overhead to zero, scaling profit margins.
- **Future:** Nimblize becomes a fully independent AI player with its own proprietary model weights.

Possible Viva Questions & Recommended Answers

- **Q:** Why is a distributed database like CockroachDB proposed for the future state?

A: As vector search traffic scales across multiple regions, a single Postgres instance becomes a single point of failure. Distributed CockroachDB guarantees continuous uptime and local data writes for fast local access.

19. Conclusion

The Phase 4 deployment establishes a deterministic, validated pipeline. By enforcing Pydantic schemas, utilizing LangGraph state machine orchestration, and gating database writes with RAGAS evaluations, we have automated competitor SEO extraction while guaranteeing zero data corruption in the production database. The proposed scaling roadmap provides a clear path to migrate to self-hosted open-source clusters, securing long-term cost sustainability and absolute data privacy.

19.1 Detailed Educational Breakdown & Viva Prep

Easy-to-Understand Explanation

In conclusion, we have built a digital assembly line. It takes raw, messy information, cleans it, extracts the facts, double-checks the work, and stores it in the warehouse. We also drew a blueprint to expand this factory so it can run on its own energy source, making it faster, cheaper, and safer for the future.

Why Nimblize Needs This

This project proves that AI can be integrated into production systems safely and reliably when bounded by standard software engineering practices.

Business & Future Impact

- **Business:** Replaces manual competitor tracking, freeing human analysts for strategic decision-making.
- **Future:** Positioned to become a leading AI-powered SEO and recommendation engine.

Possible Viva Questions & Recommended Answers

- **Q:** *Summarize the primary achievement of this implementation.*

A: We automated competitor intelligence extraction with 100% schema compliance and zero database corruption by implementing a multi-agent validation graph gated by RAGAS quality checks.

Appendix

A. API Specifications

- `POST /api/v1/pipeline/run`: Ingest raw text and run extraction.
- `POST /api/v1/b2c/recommend`: Similarity search recommendations.

B. Database Schema

PostgreSQL tables, indexes, and custom vector deletion functions.

C. Environment Variables

Configurations for OpenAI keys, database connection URLs, and JWT secrets.

D. Docker Compose Service Map

Service layouts, ports, and healthcheck commands.

E. References

LangGraph documentation, pgvector index papers, and RAGAS evaluation guides.